

Import Statements

```
In [1]: import custom_cmap
import os
import sys
from PIL import Image
from IPython.display import display
import matplotlib.pyplot as plt
import matplotlib as mpl
import pandas as pd
from moria import reduce

from pathlib import Path
from astropy.visualization import PercentileInterval

from astropy.io import fits
from astropy.visualization import LogStretch, ImageNormalize
import plotly.express as px
import numpy as np
from astropy.visualization import ImageNormalize, AsinhStretch, SqrtStretch, LogStretch, PowerStretch

plt.rcParams.update({'font.size':25})
plt.rc('text', usetex=True)
%config InlineBackend.figure_format = 'retina'
%load_ext autoreload
%autoreload 2
```

Understanding the main directory.

The data directory should be organized as follows. You can look at the sample data folder to understand the directory structure you need.

00.DATA

01.XYM

02.CMD

03.LOC_TRANS

04.PSF_EXTRACT

05.COORD_TRANS(OPTIONAL)

06.FIT

07.CALIBRATION

All the necessary scripts are included in the sample data directory.

Using the matchup files generated in "output_stacks.ipynb", a CMD file can be generated. The CMD is used to identify several important stars with colors and magnitudes similar to the target that will be used to construct a PSF model with similar CTE distortions to the target. The matchup files also select brighter stars near the target that can be used to refine the coordinate transformation and that can be used to calibrate the HST photometry to the OGLE-III catalog.

After you finish running this notebook, you will be able to use the "_flc" exposures to generate an output image stack in the F814W and F606W HST filters.

NOTE: `chmod +x program.src` is a useful command to use whenever permissions are denied for a script.

```
In [2]: directory = os.getcwd()
```

Step 0

We assume you ran the output_stacks.ipynb notebook correctly

Step 1

Run the python script to get the CMD. This step will also move the target star to the top in the MATCHUP files for the F814W and F606W folders generated in 01.XYM/F814W and 01.XYM/F606W.

We assume that IF you have a target star, you can use the 'outputq_F814W.fits' in 01.XYM/F814W to locate the pixel co-ordinates of your target. The pixel co-ordinates of your target identified in 'outputq_F814W.fits' should have a match (to the nearest decimal) in the MATCHUP files. The MATCHUP files will also give you the I magnitude needed to plot the CMD

diagram for your target. Similar steps will have to be taken to find the V magnitude of your target by navigating 01.XYM/F606W.

The rest of the arguments you give in the function "cmd_diagram" below are arbitrary and influence your plotting choice for the CMD.

reduce.cmd_diagram(directory)

```
In [17]: reduce.cmd_diagram(directory)
```

If you wish to continue with the pipeline beyond creating this CMD diagram, it is very important that your file 'NEARBY_SIM_STARS' have between 20-40 stars. This file can be found in your 02.CMD folder. If there are more than 40 stars, you should run "reduce.cmd_diagram(directory)" with tighter constraints.

The CMD diagrams are created in the 02.CMD folder. In this step we also created three important files:

1. NEARBY_SIM_STARS.XYIVB_targ - This contains the stars similar to the target to be used for the PSF model construction.
2. NEARBY_REF_STARS.XYIVB_targ - This contains stars close to the target to be used to refine the coordinate transformations.
3. NOTFAR_CAL_STARS.XYIVB_targ - This contains the stars that will be calibrated to the OGLE-III catalog. Note that there aren't very many stars that won't be blended in the OGLE-III data, so the calibration stars should be selected over a wider range of coordinates than the candidate PSF stars.

Note that the code used to identify PSF model outliers currently only allows up to 40 PSF stars, so the NEARBY_SIM_STARS.XYIVB_targ file should be kept to a maximum of 40 stars (42 lines in the file). Let us open NEARBY_SIM_STARS.XYIVB_targ to ensure that this is the case.

```
In [18]: filename_sim_stars = Path(directory).resolve()/f"02.CMD/NEARBY_SIM_STARS.XYIVB_targ"
cols = ["xu", "yu", "miu", "mvu", "psf"]
df = pd.read_csv(filename_sim_stars, sep=r"\s+", comment="#", header=None, names=cols)
```

```
In [19]: df
```

Out[19]:

	xu	yu	miu	mvu	psf
0	5067.344	5543.042	-12.567	-11.189	0
1	4905.296	5626.836	-12.862	-11.455	1
2	4917.743	5479.122	-12.355	-11.055	1
3	4934.397	5416.301	-12.513	-11.146	1
4	4936.659	5672.986	-11.220	-10.035	1
5	4950.735	5460.686	-13.895	-13.554	1
6	4955.254	5684.041	-12.816	-11.830	1
7	4992.575	5608.962	-13.698	-12.316	1
8	5003.107	5723.957	-11.499	-10.271	1
9	5019.699	5541.035	-11.850	-10.443	1
10	5020.637	5360.178	-13.027	-11.684	1
11	5025.497	5413.529	-12.078	-11.319	1
12	5038.009	5413.250	-12.188	-10.795	1
13	5060.628	5620.633	-11.098	-9.811	1
14	5083.748	5483.683	-12.246	-10.921	1
15	5084.209	5652.118	-12.437	-11.090	1
16	5107.955	5578.528	-13.424	-12.008	1
17	5116.564	5703.555	-12.493	-12.496	1
18	5117.063	5371.325	-13.819	-12.393	1
19	5131.588	5416.436	-11.974	-11.197	1
20	5160.526	5569.345	-11.879	-10.650	1
21	5163.833	5525.863	-13.666	-12.282	1
22	5178.924	5594.367	-11.444	-10.854	1

	xu	yu	miu	mvu	psf
23	5188.747	5506.271	-13.882	-12.697	1
24	5190.450	5411.579	-11.887	-11.117	1
25	5221.306	5591.853	-11.278	-10.219	1
26	5222.498	5497.895	-11.122	-10.101	1

Let us also open the file "show_cmd_targ" to inspect the CMD diagram generated around our target, along with panels showing the reference stars selected to create the "NEARBY_REF_STARS.XYIVB_targ" file

```
In [20]: import base64
from IPython.display import IFrame
pdf_path = Path(directory).resolve()/f"02.CMD/show_cmd_targ.pdf"
with open(pdf_path, "rb") as pdf_file:
    encoded_pdf = base64.b64encode(pdf_file.read()).decode("utf-8")
IFrame(f"data:application/pdf;base64,{encoded_pdf}", width=800, height=800)
```

Out [20]:



Lastly, let us open the file "show_cmd_cal" to inspect the CMD diagram generated around our target, along with panels showing the reference stars selected to create the "NEARBY_CAL_STARS.XYIVB_targ" file

```
In [21]: import base64
from IPython.display import IFrame
pdf_path = Path(directory).resolve()/f"02.CMD/show_cmd_cal.pdf"
with open(pdf_path, "rb") as pdf_file:
    encoded_pdf = base64.b64encode(pdf_file.read()).decode("utf-8")
IFrame(f"data:application/pdf;base64,{encoded_pdf}", width=450, height=450)
```

Out [21]:

